

GIT E GITHUB

Controle de Versão e Colaboração em Desenvolvimento de Software



Equipe de Desenvolvimento

 Beta Sette Zeta

 Kael Élodie Whitmore

 Zaion Alaric Zanarelli

 Professor: Marco Antonio



Syrran Tech Academy

2026

CTJ - Colégio Técnico Jundiaí - SP
Técnico em Desenvolvimento de Sistemas



○
main

○
feature

○
commit

2 ! Problema

</> No desenvolvimento de software:

- ? Como controlar mudanças no código?
- ? Como evitar perder versões anteriores?
- ? Como trabalhar em equipe sem conflitos?

💡 Surge a necessidade de **controle de versão**

👥 **85%**
de projetos precisam de
versionamento

🕒 **40%**
de tempo perdido sem controle

```
$ git status # Verifica status do repositório
```



3

O que é Git?

Definição

Git é um sistema de controle de versão distribuído que permite registrar e acompanhar todas as alterações feitas em um projeto ao longo do tempo.

 Criado por Linus Torvalds
2005

 Salva versões
Do projeto

 Distribuído
Funciona localmente

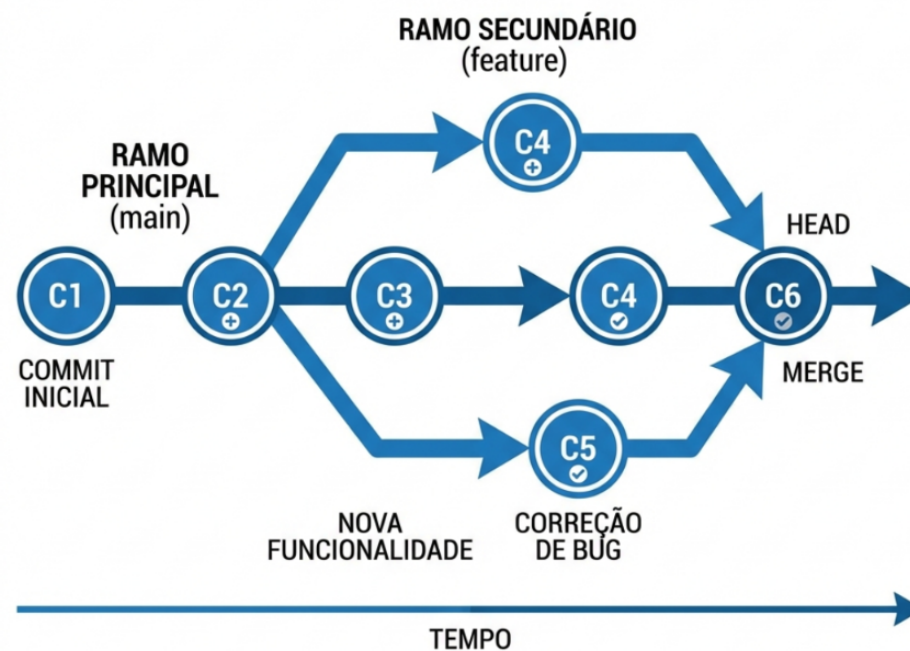
 Registro (Commit)
Cada alteração

 Cada alteração vira um  commit

 2005
Ano de criação

 90%
dos devs usam Git

REPOSITÓRIO DISTRIBUÍDO



Sistema Git: controle de versão distribuído

GitLab

Como o Git Funciona?

Linha do tempo, versionamento e histórico completo

4 / 17

🕒 Git como Linha do Tempo



- ✓ **Histórico completo:** Cada alteração vira um registro (commit) com data, autor e mensagem
- ✓ **Possibilidade de voltar:** Você pode retornar a qualquer versão anterior do projeto
- ✓ **Trabalho em equipe:** Múltiplos desenvolvedores podem trabalhar simultaneamente

💡 **Analogia:** É como salvar várias versões de um trabalho escolar, mas de forma organizada e automática. Cada "salvamento" cria um ponto de restauração que você pode acessar sempre que precisar.

🔗 Estrutura do Git

```
$ git log --oneline --graph
# Mostra histórico visual

* a1b2c3d Commit inicial
* e4f5g6h Adiciona funcionalidade X
* i7j8k9l Corrige bug Y
* m0n1o2p Versão atual
```

📄 Fluxo básico:

1. Criar repositório → 2. Adicionar arquivos → 3. Commit → 4. Push

🔄 Voltar no Tempo

Comando para reverter:

```
$ git checkout <commit-id>
```

⚠️ Cuidado: use **git reset** com cautela

5

✓ Por que usar Git?



Organização

Estrutura hierárquica clara para gerenciar projetos de forma eficiente



Segurança

Proteção contra perda de dados e backup automático



Histórico de alterações

Rastreamento completo de todas as mudanças no código



Trabalho em equipe

Colaboração eficiente entre desenvolvedores



Muito usado no mercado de tecnologia - 90% das empresas utilizam Git



95%
das empresas tech



2M+
repositórios ativos

Benefícios do Git



ORGANIZAÇÃO
TAS



SEGURANÇA
AVANÇADA



HISTÓRICO DE
ALTERAÇÕES



TRABALHO
EM EQUIPE

Benefícios do controle de versão em projetos



Instalação do Git

Configuração inicial ●



Download do Git

Versão mais recente: 2.43.0

Acesse o site oficial para baixar o Git para Windows, Mac ou Linux.

<https://git-scm.com>

Escolha a versão compatível com seu sistema operacional



Configuração Inicial

Defina seu nome e email

1 Configure seu nome de usuário

```
$ git config --global user.name "Seu Nome"
# Define o nome que aparecerá nos commits
```

2 Configure seu email

```
$ git config --global user.email "seu@email.com"
# Email associado aos commits
```

Essas configurações são globais e se aplicam a todos os repositórios



Terminal - Git Configuration

```
$ git --version
git version 2.43.0

$ git config --global user.name "João Silva"
$ git config --global user.email "joao@email.com"
$ git config --list
user.name=João Silva
user.email=joao@email.com

✓ Configuração concluída com sucesso!

$ _
```

✓ Git instalado 👤 Usuário configurado

> Comandos Básicos do Git

Principais comandos para versionamento •

</> Comandos Essenciais

1 **git init**
Inicializa um repositório Git

2 **git add .**
Adiciona arquivos para staging

3 **git commit -m**
Cria commit com mensagem

4 **git status**
Verifica estado dos arquivos

5 **git log**
Mostra histórico de commits

6 **git clone**
Clona repositório remoto

💡 **Dica:** Use `git status` antes de commit para verificar o estado dos arquivos

Terminal - Git Commands

```
$ git init
Initialized empty Git repository in /project/.git/
$ git add .
Adding files to staging area...
$ git commit -m "primeira versão"
[main (root-commit) a1b2c3d] primeira versão
$ git status
On branch main, nothing to commit
$ git log --oneline
a1b2c3d primeira versão
```

> Exemplo Prático

Criando versões do projeto ●



Exemplo Inicial

Primeira versão do projeto

1 Inicializar repositório Git

```
$ git init  
# Inicializa um novo repositório Git
```

2 Adicionar arquivo ao staging

```
$ git add index.html  
# Adiciona o arquivo para o próximo commit
```

3 Criar primeiro commit

```
$ git commit -m "primeira versão"  
# Salva a primeira versão do projeto
```



Depois de Modificar

Atualização do projeto

1 Adicionar todas as alterações

```
$ git add .  
# Adiciona todos os arquivos modificados
```

2 Criar commit de atualização

```
$ git commit -m "atualização"  
# Salva as alterações no histórico
```

i Cada commit cria uma nova versão do projeto



✓ Criamos versões do projeto

O que é GitHub?

Plataforma de Desenvolvimento Colaborativo

GitHub é uma **plataforma online** que utiliza o Git para versionamento de código. Funciona como um "Google Drive de código", permitindo armazenar projetos, colaborar com outros desenvolvedores e gerenciar versões de forma distribuída.

Repositórios na nuvem

Colaboração

Versionamento

Segurança

Baseado no Git - Sistema de controle de versão distribuído

Analogia: "Google Drive de código" - Armazene e compartilhe seu código

GitHub como plataforma de colaboração em nuvem

GITHUB: PLATAFORMA DE NUVEM



Plataforma GitHub: repositórios, colaboração e versionamento

10

🎯 Para que serve o GitHub?

GitHub é uma plataforma online que revoluciona a forma como desenvolvedores trabalham. Oferece ferramentas para colaboração, versionamento e compartilhamento de código de forma eficiente.

☁ É como um **"Google Drive de código"** - armazena, versiona e colabora em projetos

 **50M+**
desenvolvedores

 **100M+**
repositórios

 **Colaboração Eficiente**
Trabalhe com múltiplos desenvolvedores simultaneamente

 **Visibilidade Global**
Seus projetos acessíveis de qualquer lugar

 **Deploy Automático**
Integração com serviços de hospedagem

 **Trabalhar em Equipe**
Colaboração simultânea com controle de versões

 **Compartilhar Projetos**
Distribuição fácil de código e recursos

 **Organizar Ideias**
Gestão de issues e projetos

 **Construir Portfólio**
Exposição profissional de projetos

11

Criando conta no GitHub

1 Acesse o site oficial

Visite **<https://github.com>** para iniciar o processo de criação de conta.

 <https://github.com>

2 Criar conta gratuita

Preencha seus dados (nome, email, senha) e verifique seu e-mail para ativar a conta.

3 Criar repositórios

Após login, clique em "New" para criar seu primeiro repositório de projeto.

✓ Benefícios

- ✓ Conta gratuita
- ✓ Repositórios ilimitados
- ✓ Compartilhar projetos
- ✓ Colaboração em equipe



GitHub - Plataforma de desenvolvimento

 É como um "Google Drive de código"



Integração Git + GitHub

Conectando local com remoto ●



git clone URL

Copia um repositório remoto para sua máquina local

```
$ git clone https://github.com/usuario/projeto.git  
# Cria uma cópia local do repositório remoto
```

Clone: Baixa todo o histórico do projeto e cria uma cópia local



git push origin main

Envia alterações locais para o repositório remoto

```
$ git push origin main  
# Envia commits locais para o branch main
```

Push: Envia alterações locais para o repositório remoto



git pull origin main

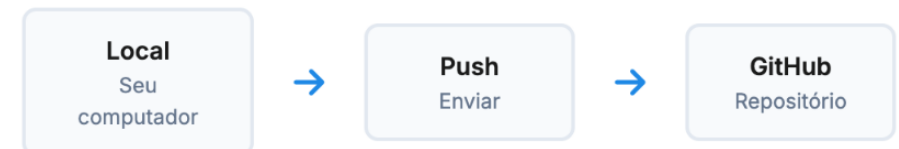
Atualiza projeto local com alterações do repositório remoto

```
$ git pull origin main  
# Atualiza o projeto local com as mudanças remotas
```

Pull: Atualiza projeto local com alterações do repositório



Fluxo de Trabalho

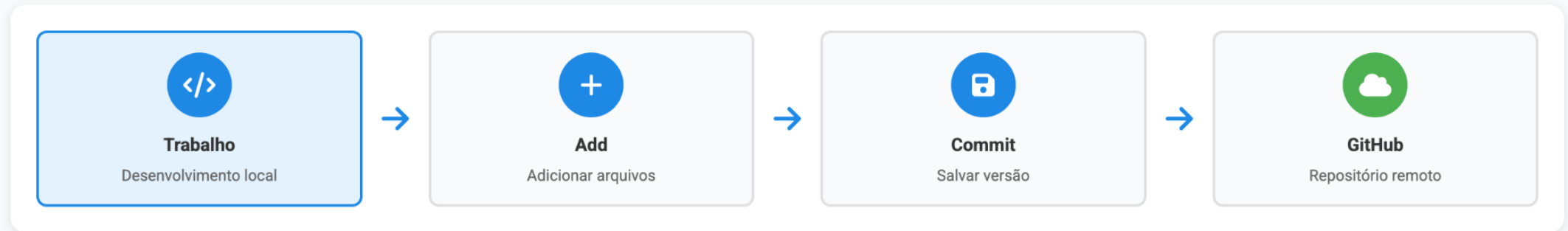


✓ Conectado ↻ Sincronizado

Fluxo do Git

13 / 17

Processo completo: Trabalho → Add → Commit → Push → GitHub



Comandos Principais

```
$ git add . # Adiciona arquivos
$ git commit -m "mensagem" # Cria commit
$ git push origin main # Envia para GitHub
```

Ciclo Contínuo

✓ Desenvolvimento contínuo:

O fluxo se repete a cada modificação, mantendo o histórico de versões atualizado.

 **Dica:** Execute `git status` antes de cada commit para verificar o estado dos arquivos.

14

Aplicação Prática

No projeto escolar:

- ✓ Organização do código
- ✓ Trabalho em grupo
- ✓ Controle de versões

 Cada aluno pode contribuir ao projeto

 100%
de colaboração

 24/7
acesso ao código

```
$ git push origin main # Envia alterações para o repositório
```





Benefícios do Git e GitHub

- ✓ **Git organiza o desenvolvimento** - Permite salvar versões do projeto e rastrear todas as alterações
- ✓ **GitHub facilita colaboração** - Permite trabalhar em equipe de forma eficiente
- ✓ **Essencial no mercado de trabalho** - Muito usado por empresas de tecnologia

95%

Adoção no mercado

2M+

Repositórios ativos

100%

Essencial para devs

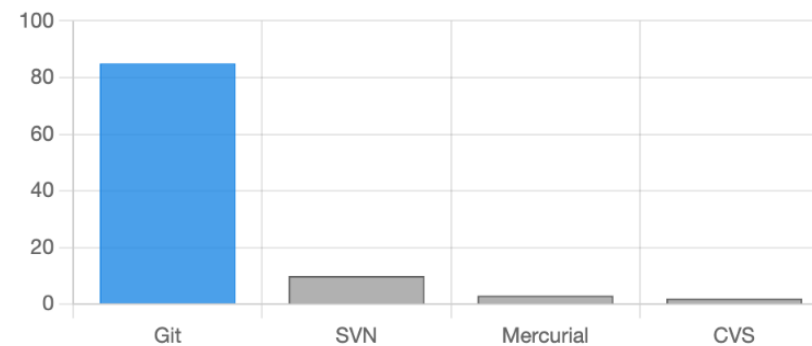


Aprender Git = aprender a trabalhar em equipe

O controle de versão é fundamental para o desenvolvimento colaborativo



Impacto no mercado





O que acontece se **não controlarmos as versões de um projeto?**



Perda de código



Conflitos de versão



Desperdício de tempo



Erros irreversíveis

Reflexão: O controle de versão é fundamental para garantir a organização, segurança e colaboração eficiente em projetos de desenvolvimento de software. Sem ele, o trabalho em equipe se torna caótico e produtividade é comprometida.



Pense sobre como o Git pode resolver esses problemas no seu próximo projeto

 Bibliografia **Pro Git**
CHACON, Scott **Version Control with Git**
LOELIGER, Jon**CHACON, Scott. Pro Git**

Livro completo sobre Git, cobrindo desde conceitos básicos até tópicos avançados de versionamento.

git-scm.com/book

 Links Úteis git-scm.com github.com**LOELIGER, Jon. Version Control with Git**

Guia prático sobre controle de versão com Git para desenvolvedores.

git-scm.com/doc